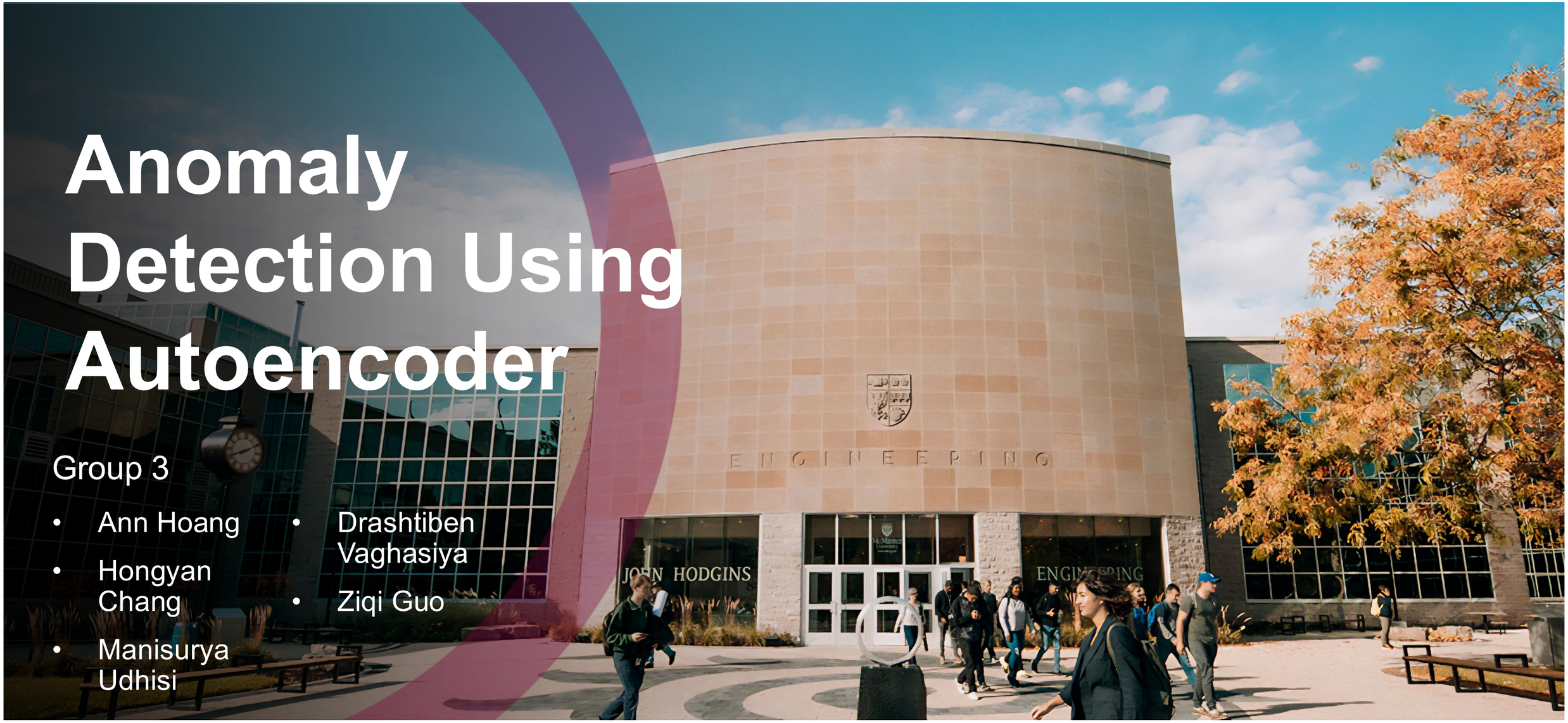
Anomaly Detection Using Autoencoder



Group 3

- Ann Hoang
- Hongyan Chang
- Manisurya Udhisi
- Drashtiben Vaghasiya
- Ziqi Guo

McMaster University | Engineering

July 31, 2026

Table of Content

McMaster University | Engineering

Section 1

Problem Statement

Drashtiben Vaghasiya

McMaster University | Engineering

Problem Statement

- **Cyber attacks are increasing** => network security more challenging.
- **Traditional intrusion detection systems** struggle to detect **new (zero-day) attacks** and are affected by **class imbalance**.

=> Solution: Train an **unsupervised autoencoder** on normal traffic to detect anomalies through reconstruction error.

- **Objective:** Compare Basic, Sparse, and Deep Autoencoders to identify the most effective model for detection accuracy and lower false alarms.

Autoencoder Model & Theory

- **Encoder (f_θ):** Compresses 74 features into 16 latent dimensions

$$z = f_\theta(x) = \sigma(W_e x + b_e)$$

- **Decoder (g_ϕ):** Reconstructs original features from latent code

$$\hat{x} = g_\phi(z) = \psi(W_d z + b_d)$$

- **Loss Function (MSE):** $\mathcal{L}_{\mathbf{MSE}}(x, \hat{x}) = \frac{1}{d}\sum_{j=1}^{d}(x_j - \hat{x}_j)^2$

- **Decision Rule:** Calibrate threshold τ on normal data $\rightarrow$ Flag sample as an anomaly if $L > \tau$

KDD Cup 1999 Dataset Overview

- **Dataset Scale:** 494,021 connection records | 41 attributes (38 numeric, 3 categorical).
- **4 Feature Categories:**
  - *Basic (1–9):* TCP/IP headers (protocols, byte counts).
  - *Content (10–22):* Payload checks (failed logins, root shell access).
  - *Time Traffic (23–31):* 2-second window connection stats.
  - *Host Traffic (32–41):* 100-connection historical destination stats.

Attack Categories

Category	Threat Mechanism	Examples
DoS	Floods resources to crash services	neptune, smurf
Probe	Scans network to map vulnerabilities	satan, ipsweep
R2L	Remote unauthorized local access	guess_passwd, imap
U2R	Local user to superuser root escalation	buffer_overflow, rootkit

- **Key Challenge:** DoS/Probe show large anomalies; R2L/U2R stealthily mimic normal traffic.

Section 2

Data Preprocess
Ziqi Guo

McMaster University | Engineering

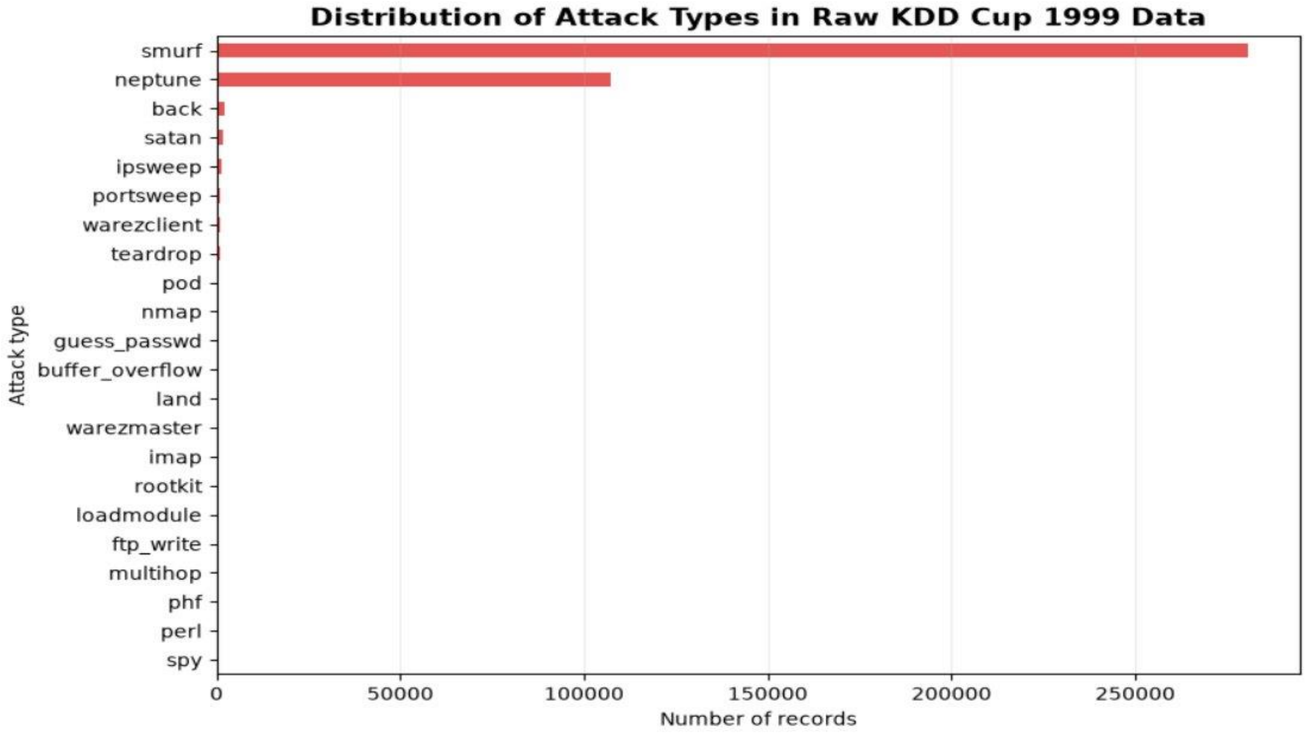
Goal: Network Anomaly Detection Using Autoencoders

- **Dataset**: KDD Cup 1999 network traffic data
- **Goal**: Detect abnormal traffic records using reconstruction error
- **Main idea**: Train an autoencoder then identify records that are poorly reconstructed

Data Processing

	duration	protocol_type	service	flag	src_bytes	dst_bytes	count	srv_count	same_srv_rate	dst_host_count	label
143854	0	icmp	tim_i	SF	564	0	1	1	1.0	2	normal
345951	0	icmp	tim_i	SF	564	0	1	1	1.0	2	pod

- **Raw Data Issues:**
  - Duplicate records, Conflicting labels, Imbalanced attack types
- **Cleaning Strategy:**
  - Remove duplicate records
  - Remove records with conflicting labels
  - Separate normal traffic and attack traffic



Preprocessed Features

- **Feature Transformation:**
  - 74 Processed input feature
  - Numerical features scaled to [0,1] (lop1p, Min-Max Scaling)
- **Data Split:**
  - Training normal data -> model training
  - Validation normal -> model selection
  - Calibration normal -> threshold calibration
  - Anomaly -> evaluation

Section 3

Basic Autoencoder
Ziqi Guo

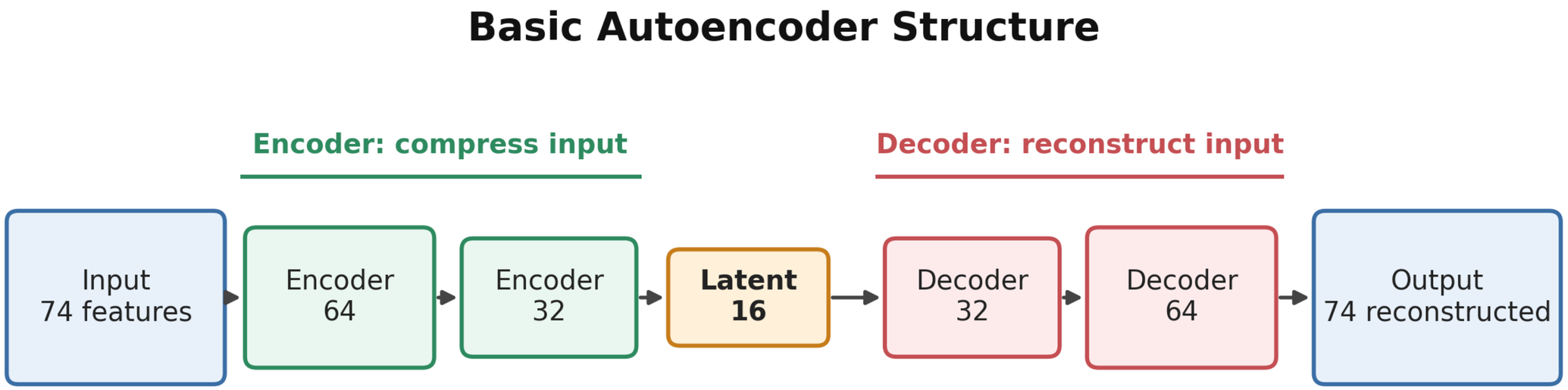
McMaster University | Engineering

Hyperparameter Selection

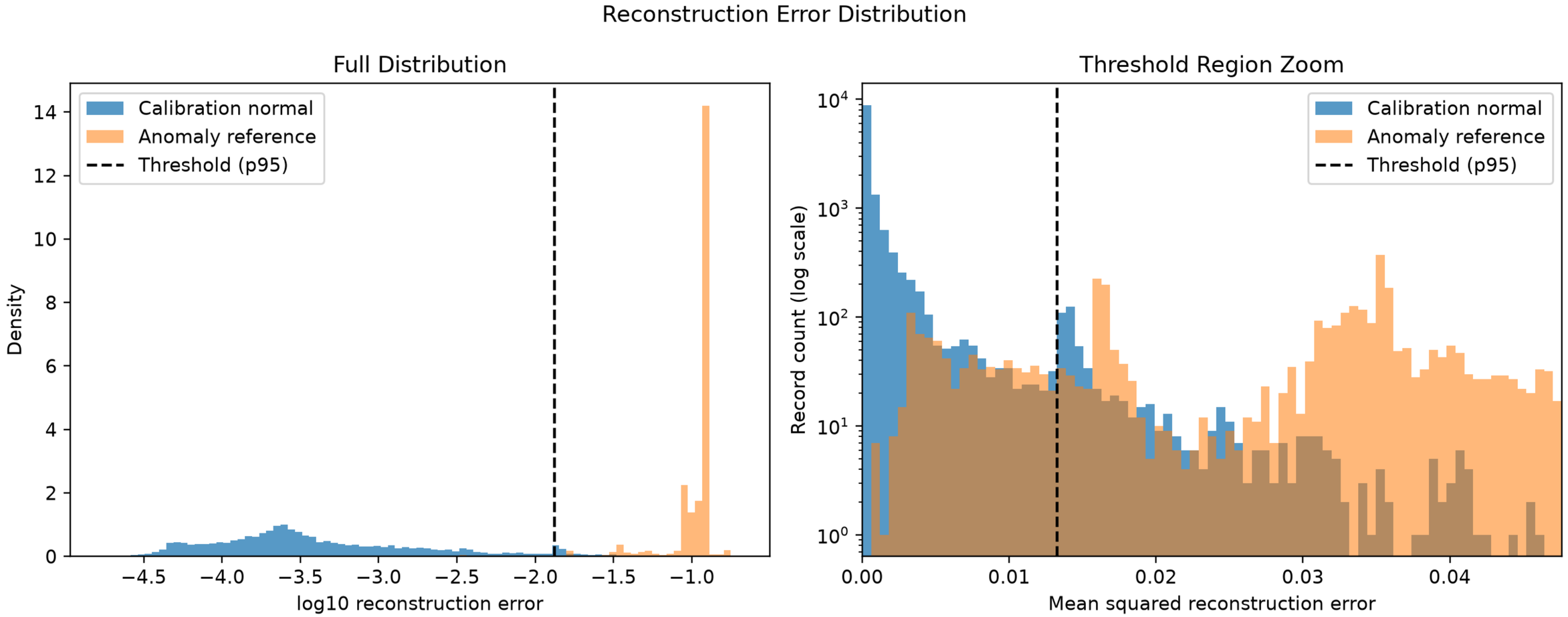
- **Compared Configurations**
  - Hidden layer sizes, Latent dimension, learning rate, weight decay
- **Selection Criteria**
  - Reconstruction loss, temp f1 score, early stop

Basic Autoencoder Structure

- **Final Model Architecture**



Result & Evaluation of Basic Autoencoder



Section 4

Sparse Autoencoder

Hongyan Chang & Drashtiben Vaghasiya

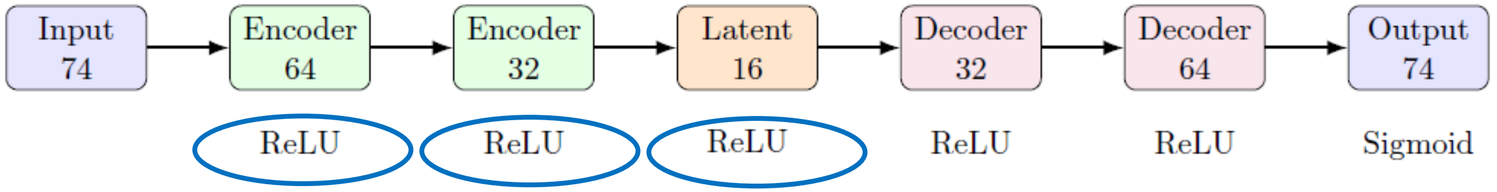
Sparse Autoencoder

- **What is a sparse autoencoder?**
  - **Standard autoencoder:** Compresses all input data into a dense, packed code.
  - **Sparse autoencoder:** Forces most neurons in the hidden layer to stay inactive
- **Why we use it?**
  - **Learns Better Features:** By turning off most neurons, the model is forced to focus only on a few, highly **specific patterns** of normal traffic.
  - **Spots Sneaky Attacks:** Normal traffic is reconstructed easily with low error. Unknown or rare attacks (like R2L or U2R) fail to reconstruct, creating a high error score that makes them easy to flag.

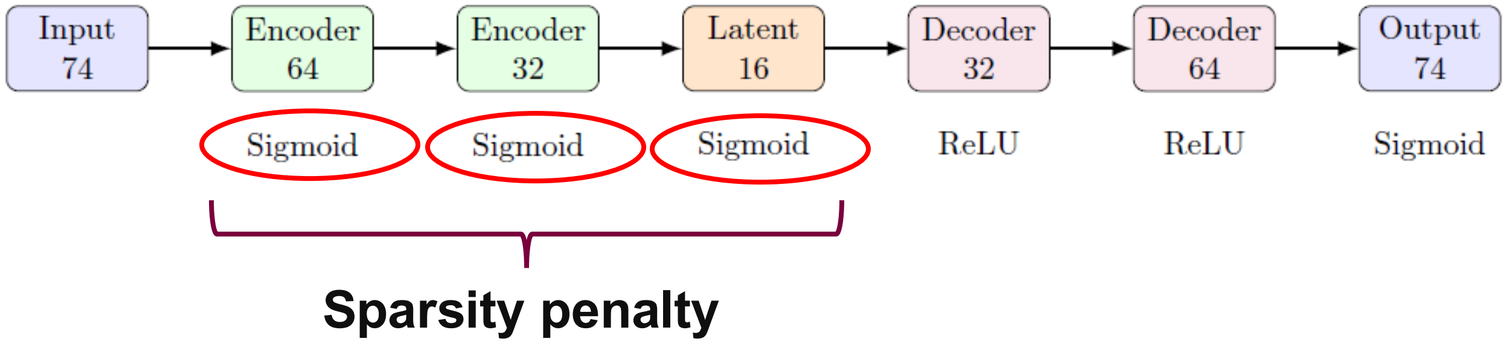
Sparse Autoencoder Structure

- The layer sizes mirror the final basic autoencoder (74 -> 64 -> 32 -> 16 ->32 -> 64 -> 74).
- The only structural change is that the encoder uses **Sigmoid**.
- The training loss adds a KL-divergence **sparsity penalty** on the encoder activations.

Basic Autoencoder



Sparse Autoencoder



Sparse Autoencoder Implement Pipeline

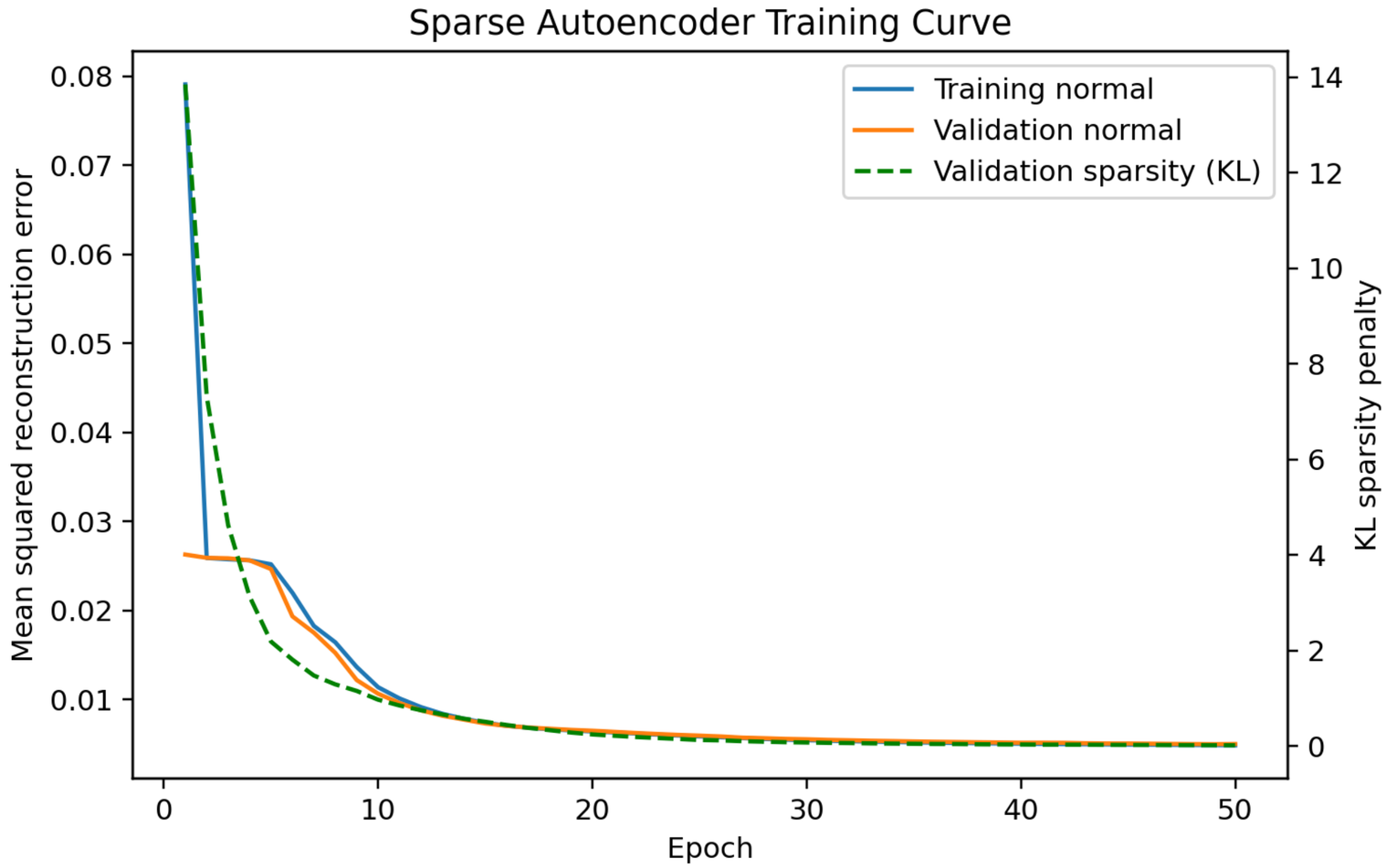
- **We use the same pipeline as the basic autoencoder**

- **Furthermore, two additional scripts implement the hyperparameter studies**
  - sparsity weights: **β = 0.001**
  - sparsity target: **ρ = 0.1**

The detailed result refer to the result section.

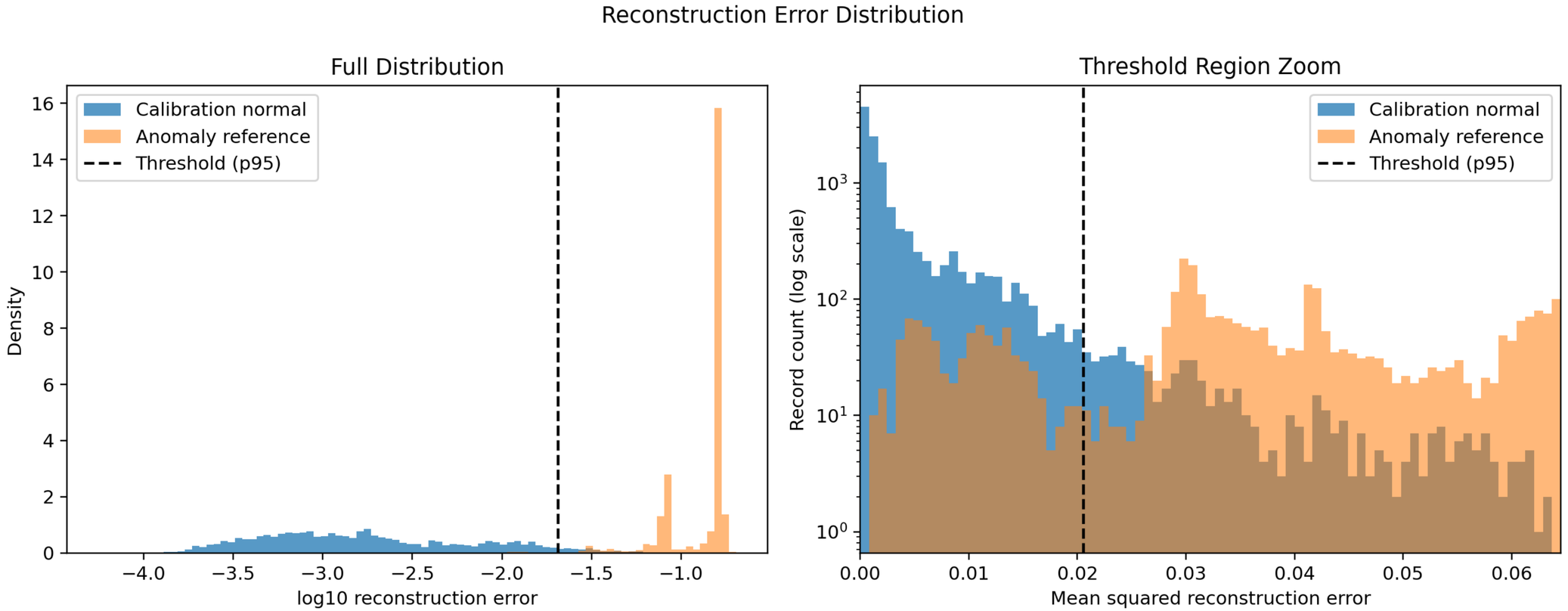
Results of Sparse Autoencoder

- **Training curve**
  - Training and validation reconstruction loss
  - Sparsity penalty



Results of Sparse Autoencoder

- **Reconstruction error distribution curve**

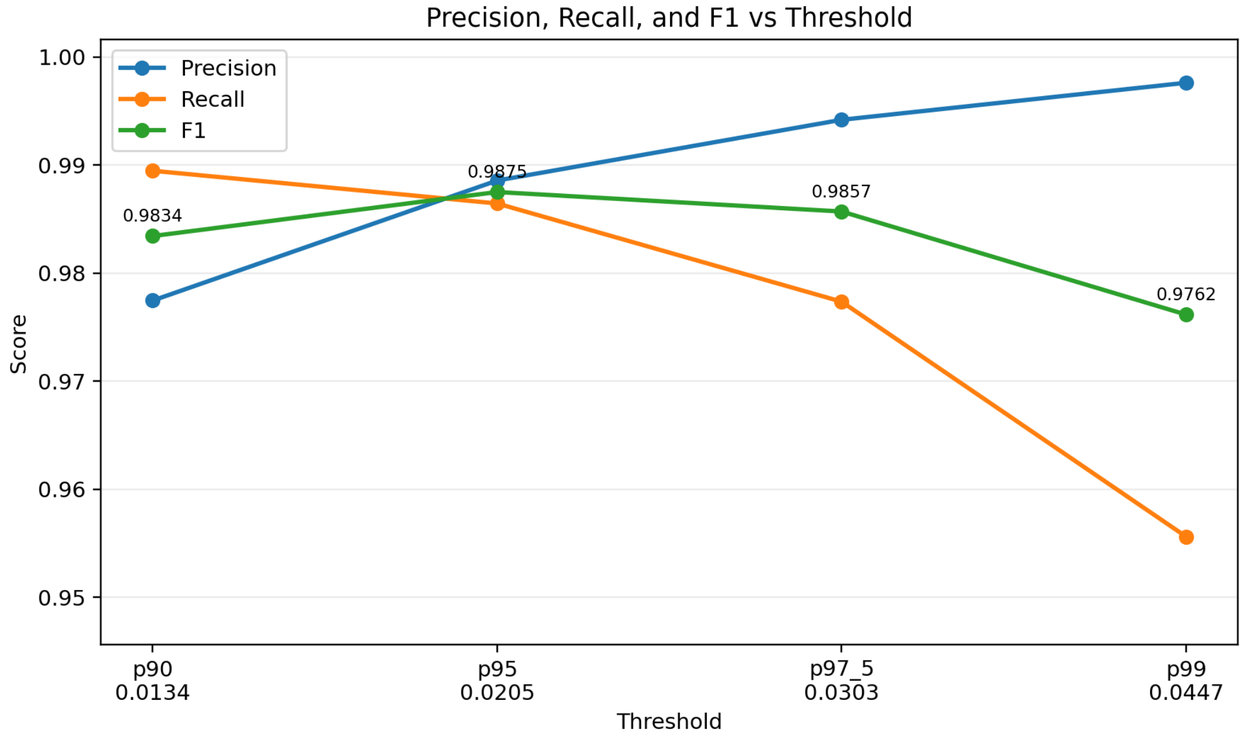


Results of Sparse Autoencoder

- **Result of sparse autoencoder under different percentile thresholds**

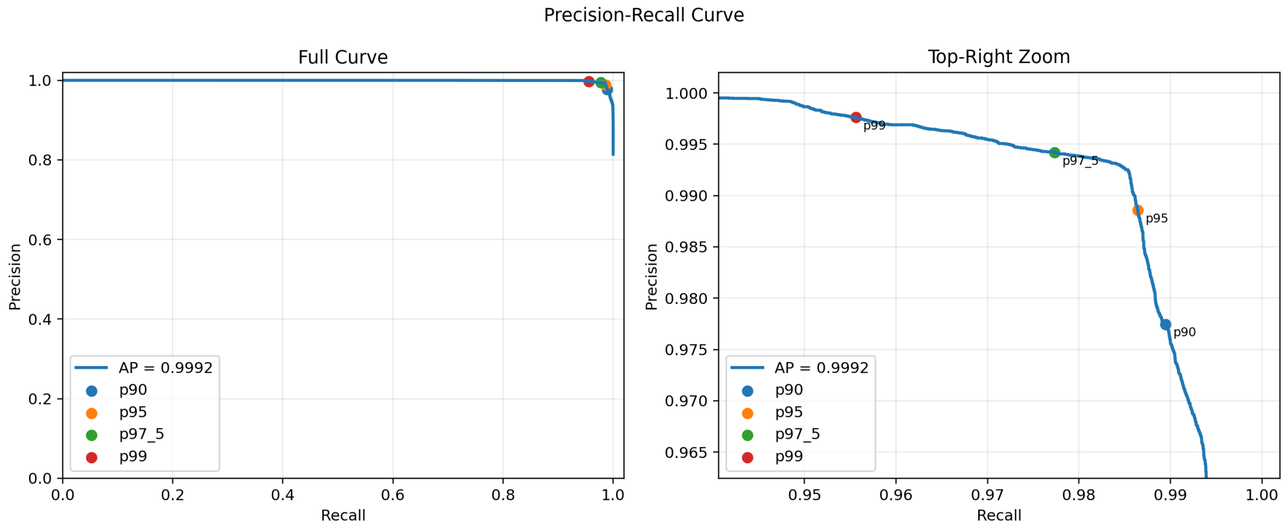
Threshold	Value	Precision	Recall	F1	Accuracy
p90	0.0134	0.9775	0.9895	0.9834	0.9728
p95	0.0205	0.9886	0.9864	0.9875	0.9797
p97.5	0.0303	0.9942	0.9774	0.9857	0.9769
p99	0.0447	0.9976	0.9556	0.9762	0.9620

- **Precision, recall, and F1 score curve**

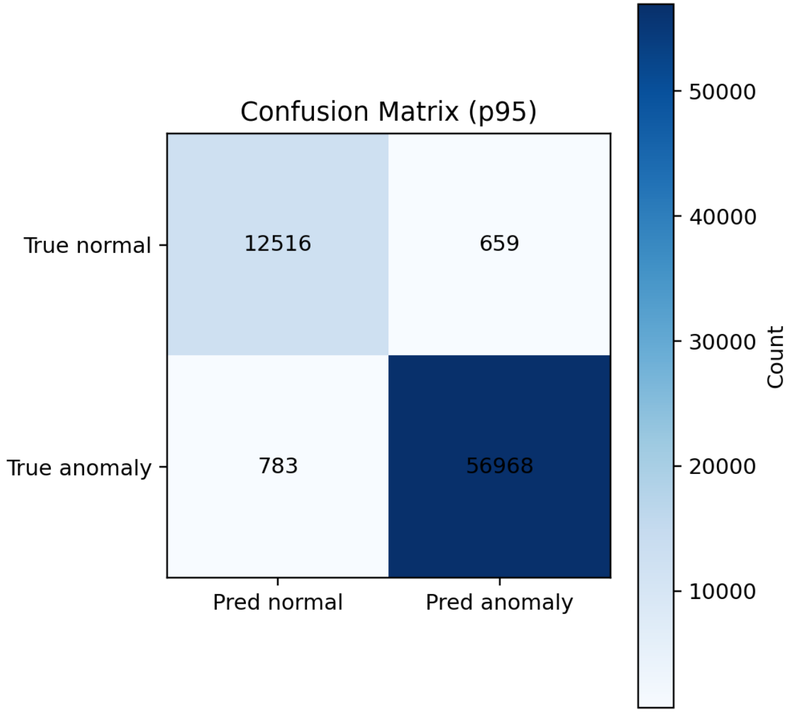


Results of Sparse Autoencoder

- **Precision - recall curve**

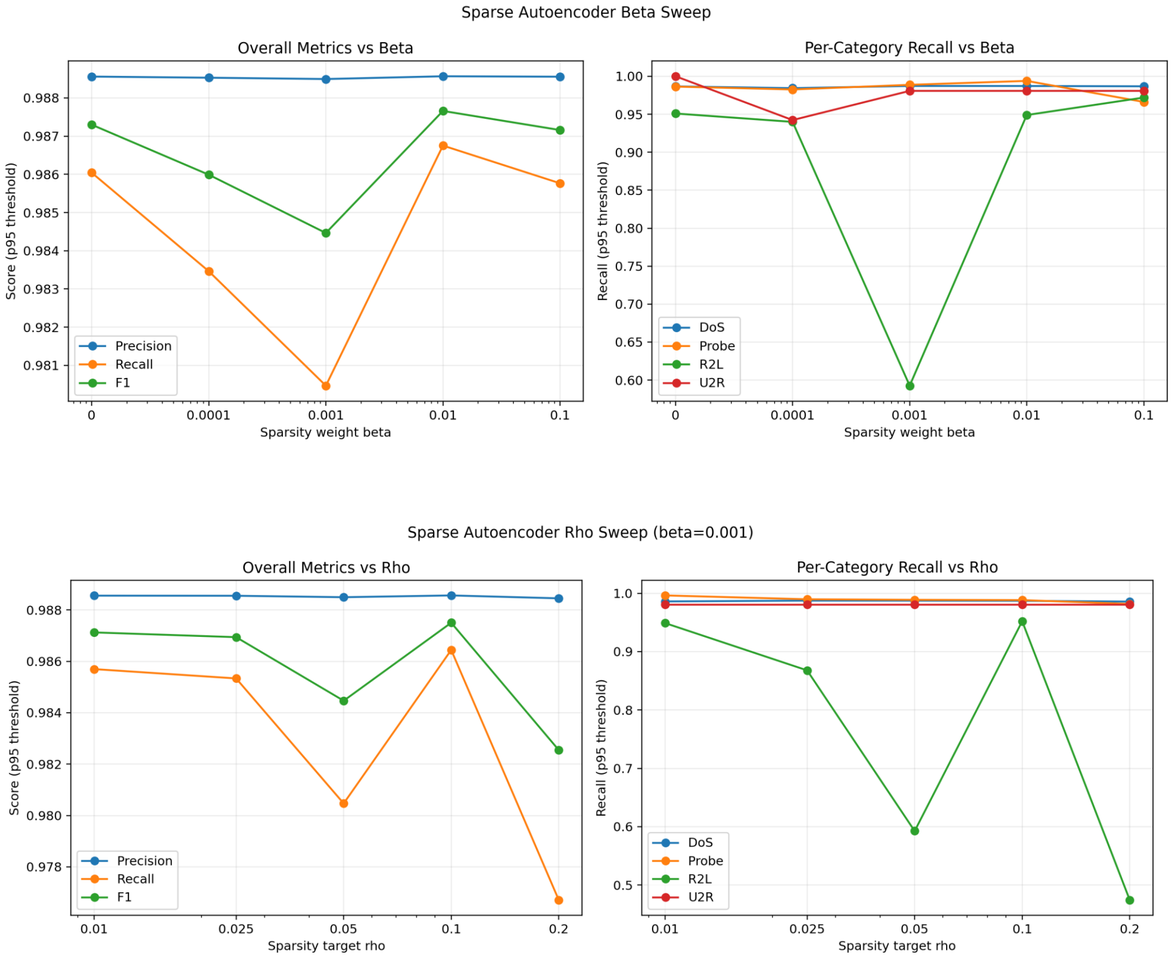


- **Confusion matrix**
  - Using the p95 threshold



Results of Sparse Autoencoder – Hyperparameter Study

- **Sparsity weight β study (fixed ρ = 0.05)**
- **Sparsity target ρ study (fixed β = 0.001)**
- **Key takeaways and limitation**
  - Final config **β = 0.001, ρ = 0.1**
  - At this capacity, detect mainly comes from autoencoder architecture and reconstruction error score
  - Sparsity is a mild, config-dependent regularizer
  - R2L recall is most sensitive and remains the main target for future improvement



Section 5

Deep Autoencoder

Ann Hoang

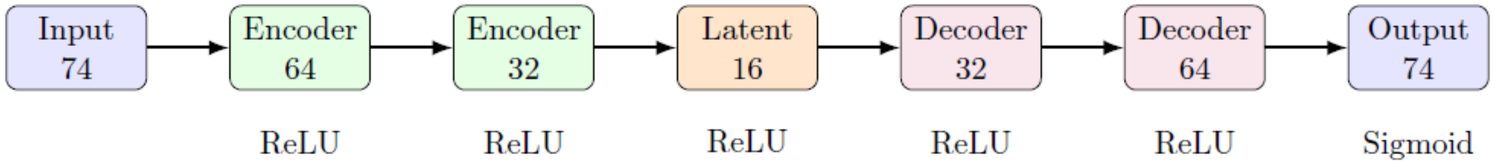
Deep Autoencoder

- **What is a deep autoencoder?**
  - **Deep autoencoder**: Uses multiple hidden encoding and decoding layers to learn hierarchical and more complex representations of the input data.
- **Why we use it?**
  - **Captures more complex patterns** by using multiple hidden layers to learn hierarchical representations of network traffic.
  - **Expected to improve anomaly detection** by modeling normal traffic more effectively, although performance depends on the dataset.

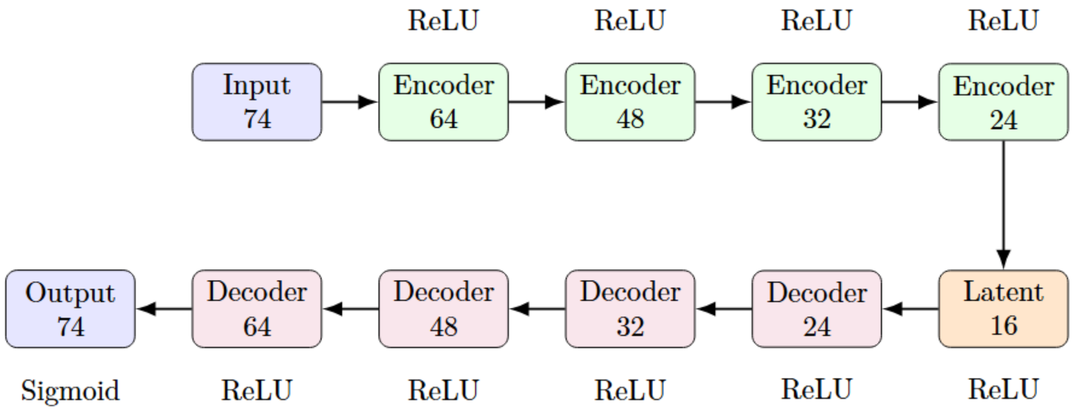
Deep Autoencoder Structure

- **Inherited the best hyperparameters from the basic autoencoder:** learning rate = 1e-3, latent space = 16, and the same activation functions.
- Added two hidden layers to both the encoder and decoder, creating a deeper network

Basic Autoencoder



Deep Autoencoder



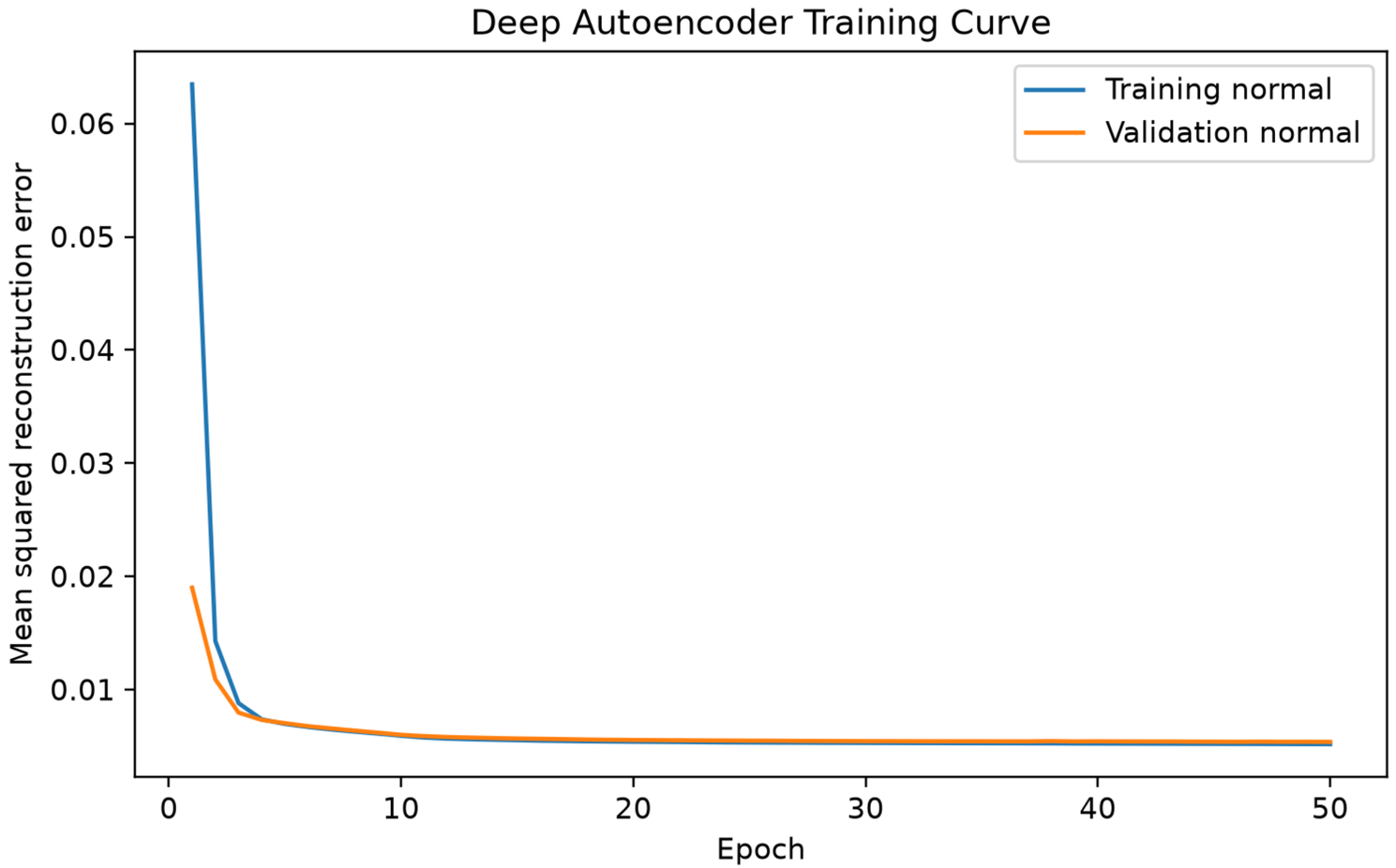
Deep Autoencoder Implement Pipeline

- **Used the same pipeline as the basic and sparse autoencoders**

- **Key difference:** Deep autoencoder implemented in **TensorFlow**; basic and sparse autoencoders implemented in **PyTorch** to gain experience with different frameworks.

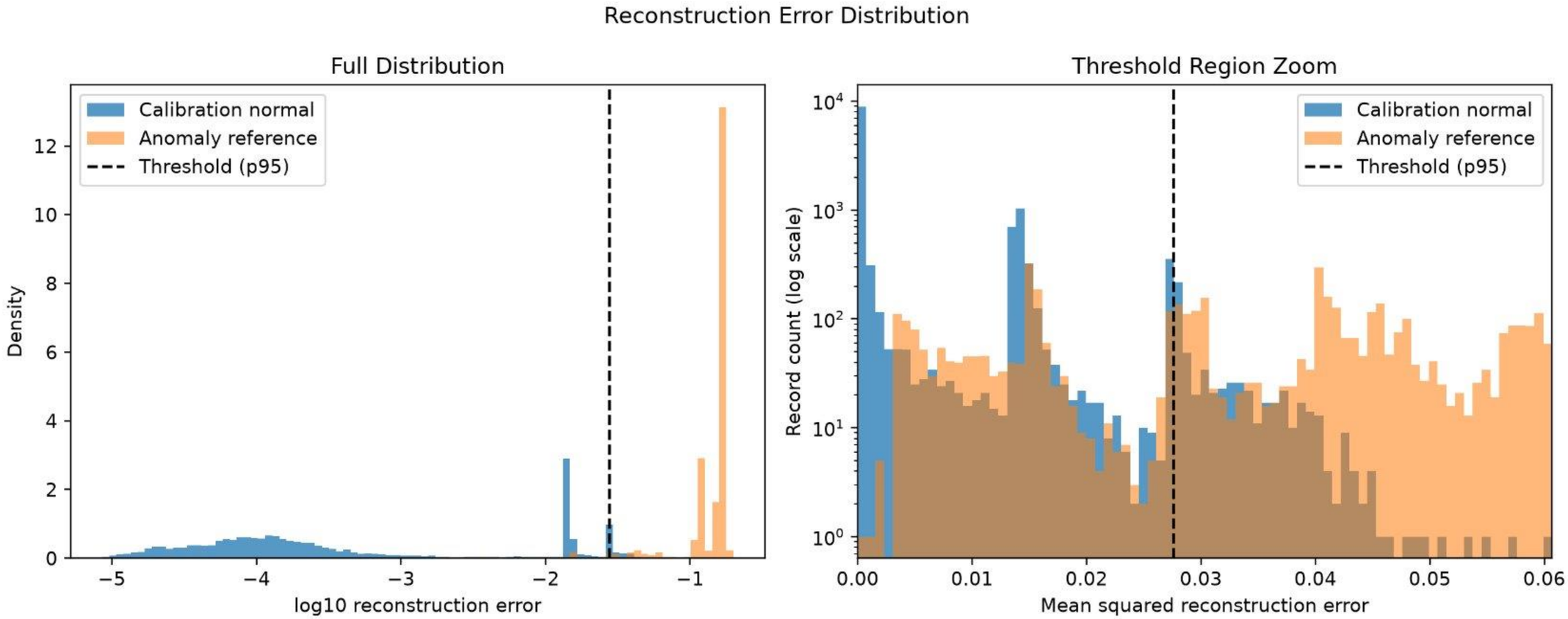
Results of Deep Autoencoder

- **Training curve**
  - Training and validation reconstruction loss



Results of Deep Autoencoder

- **Reconstruction error distribution curve**

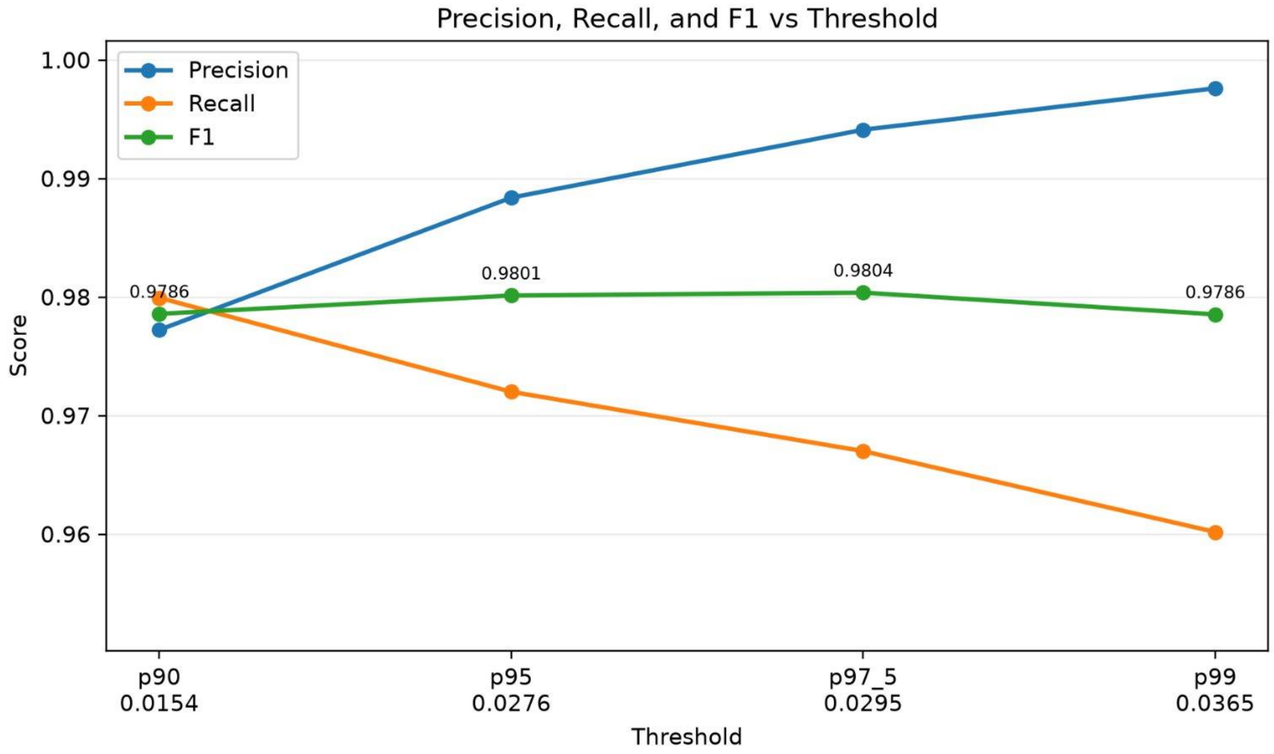


Results of Deep Autoencoder

- **Result of deep autoencoder under different percentile thresholds**

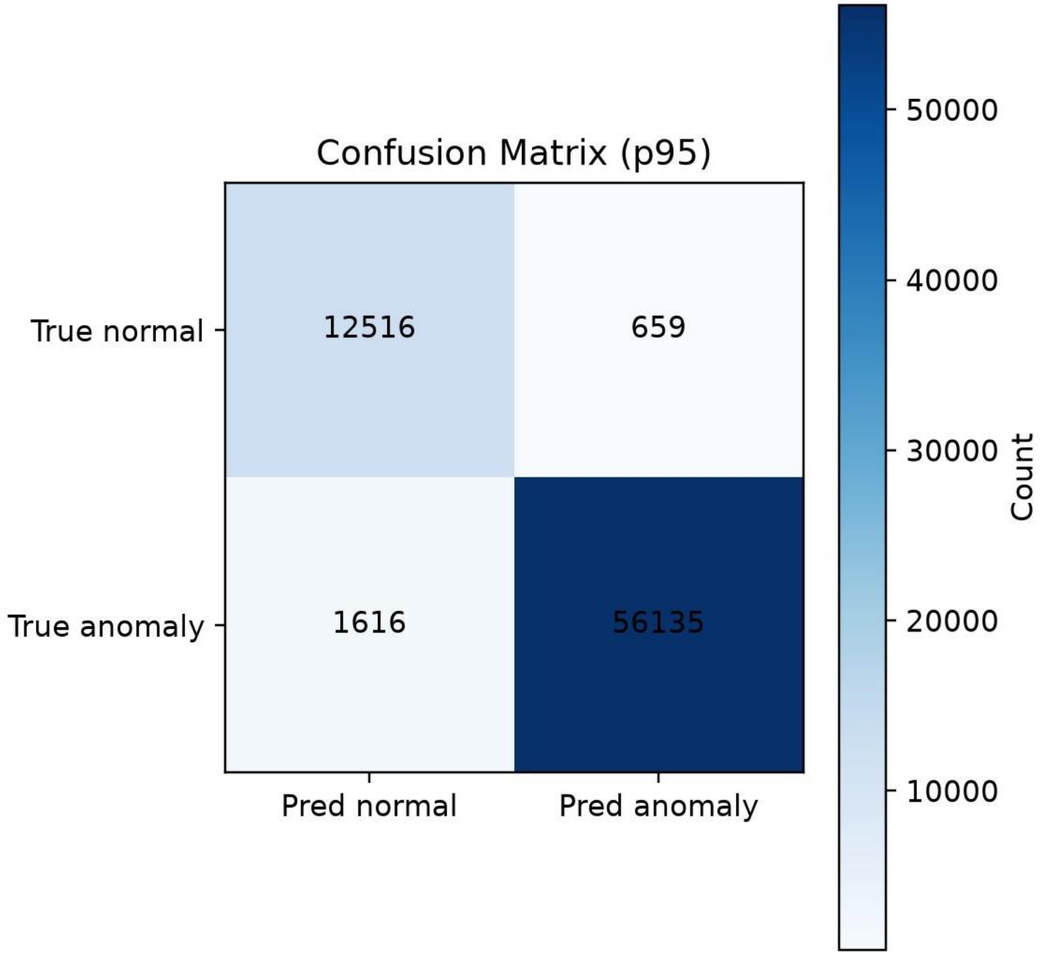
Threshold	Value	Precision	Recall	F1	Accuracy
p90	0.0144	0.9774	0.9864	0.9819	0.9703
p95	0.0166	0.9885	0.9767	0.9825	0.9717
p97.5	0.0243	0.9942	0.9744	0.9842	0.9745
p99	0.0305	0.9976	0.9630	0.9800	0.9680

- **Precision, recall, and F1 score curve**



Results of Deep Autoencoder

- **Confusion matrix:** using the p95 threshold
  - Accurate detection of normal traffic
  - Lower anomaly detection performance than the basic and sparse autoencoders



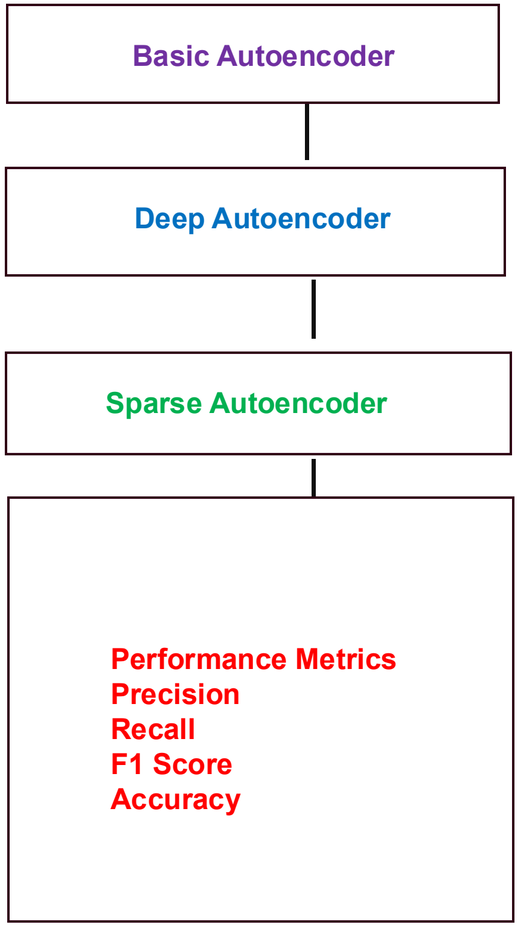
Section 6

Model Evaluation and Comparison

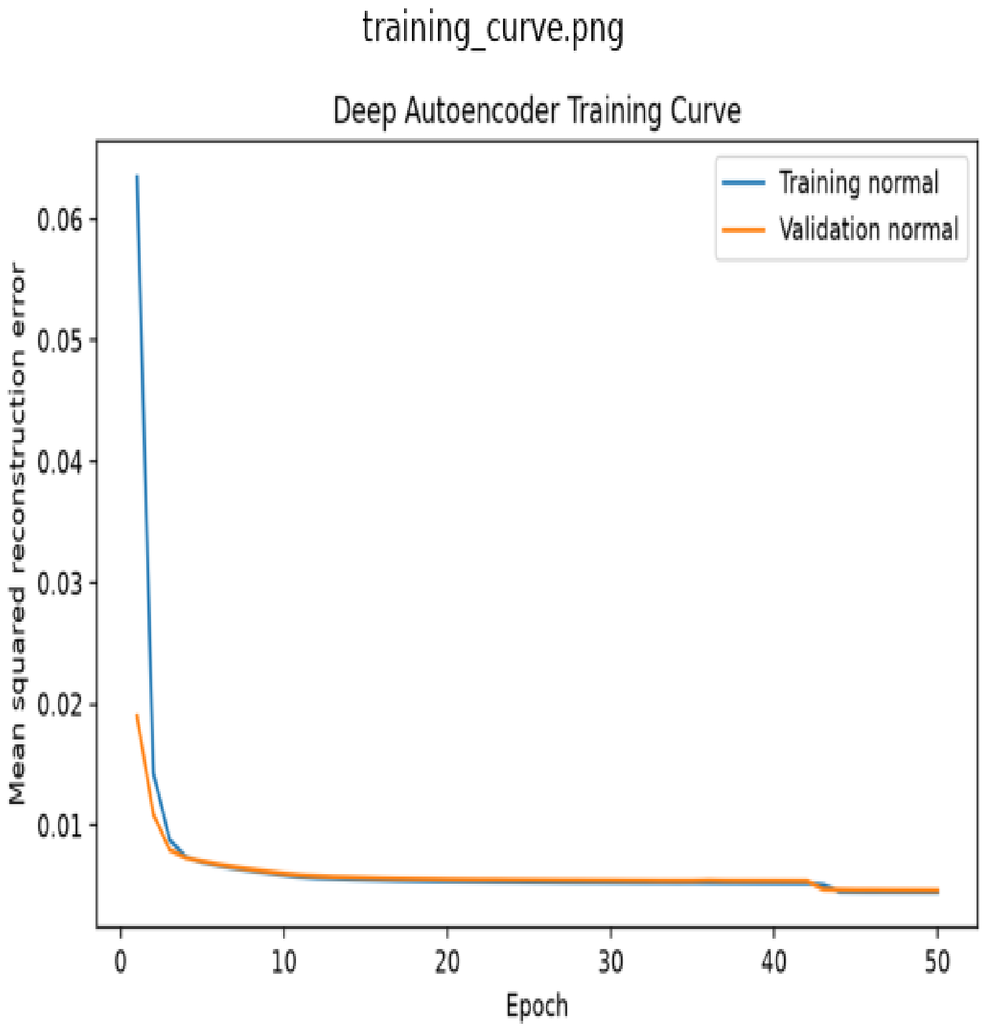
Manisurya Udhisi

Model Comparison

Performance Comparison of Autoencoder Models



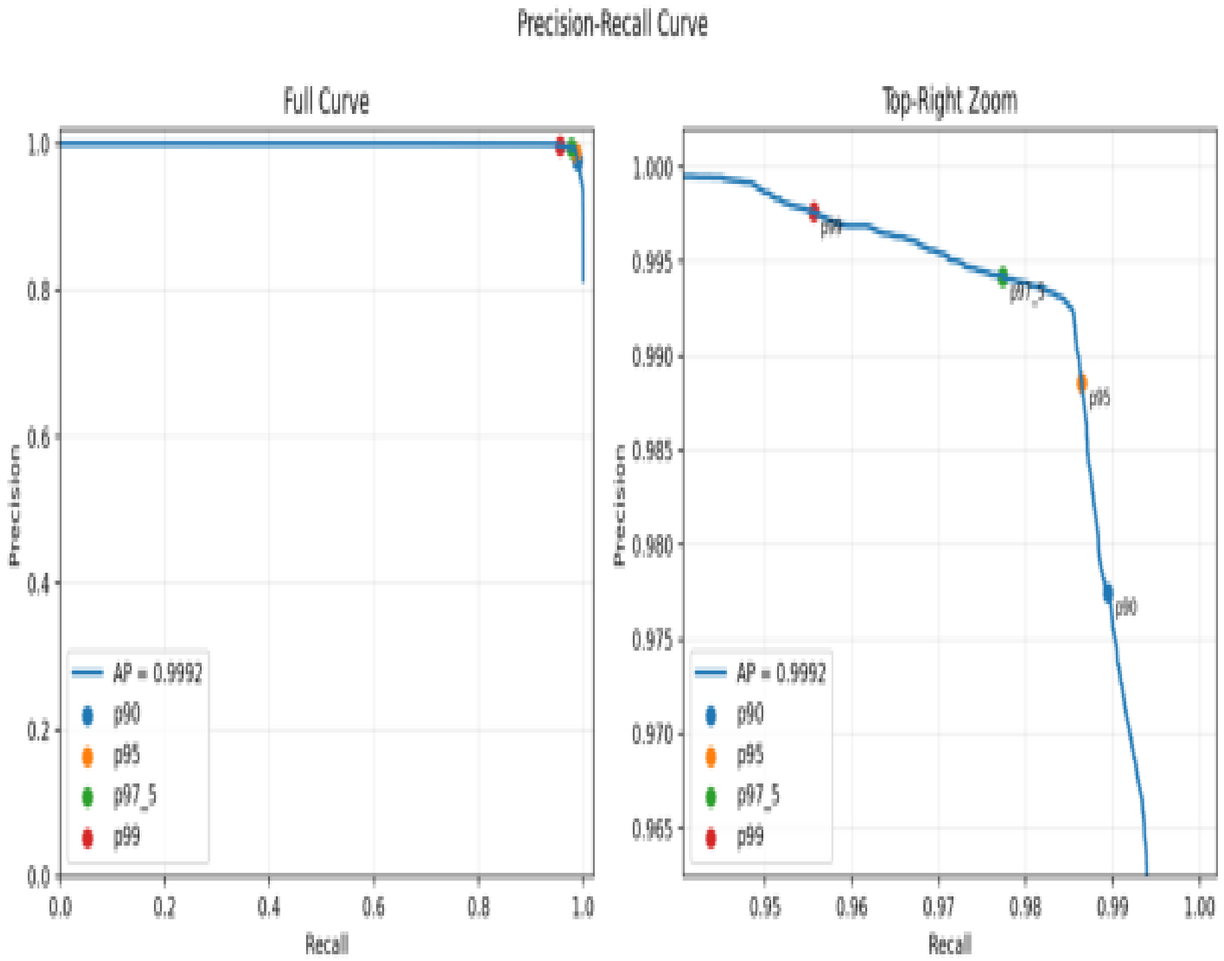
training_curve.png



Performance Comparison

Evaluation Results of Autoencoder Models

Model	Precision	Recall	F1 Score	Accuracy
Basic Autoencoder	**0.9886**	**0.9866**	**0.9876**	**0.9798**
Deep Autoencoder	0.9885	0.9767	0.9825	0.9717
Sparse Autoencoder	0.9886	0.9864	0.9875	0.9797

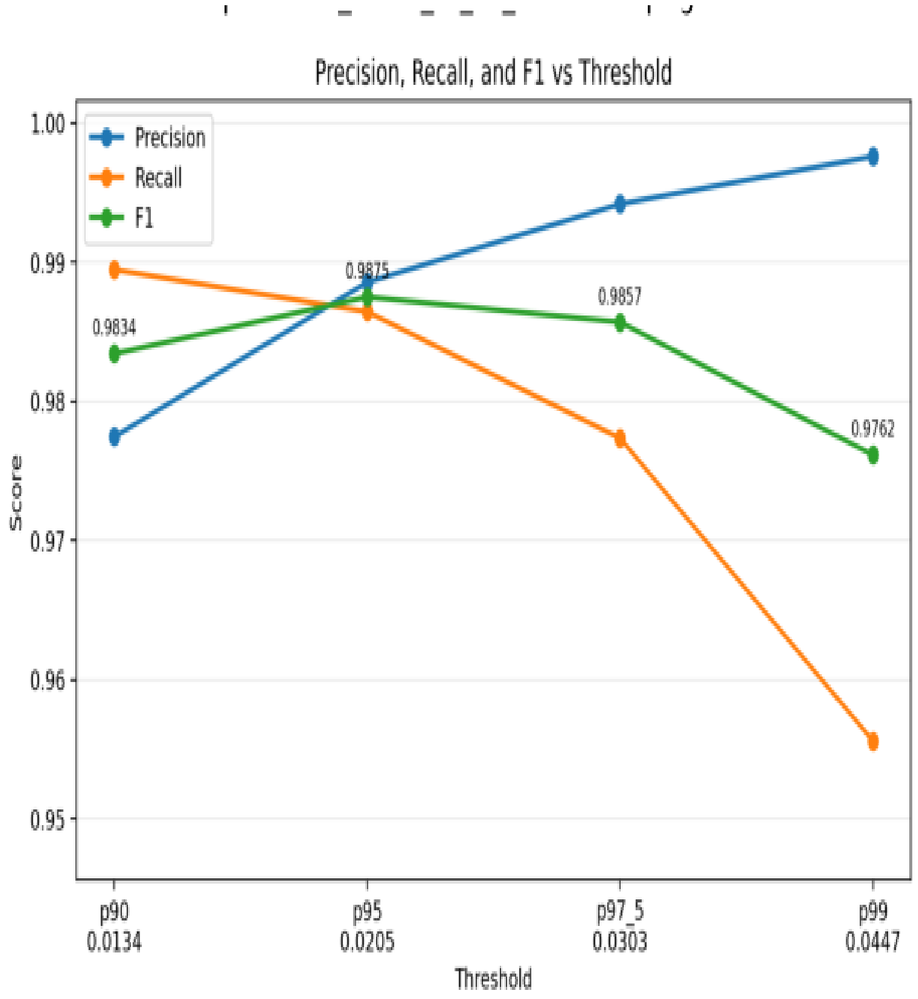


Results Visualization

Performance Analysis of Autoencoder Models

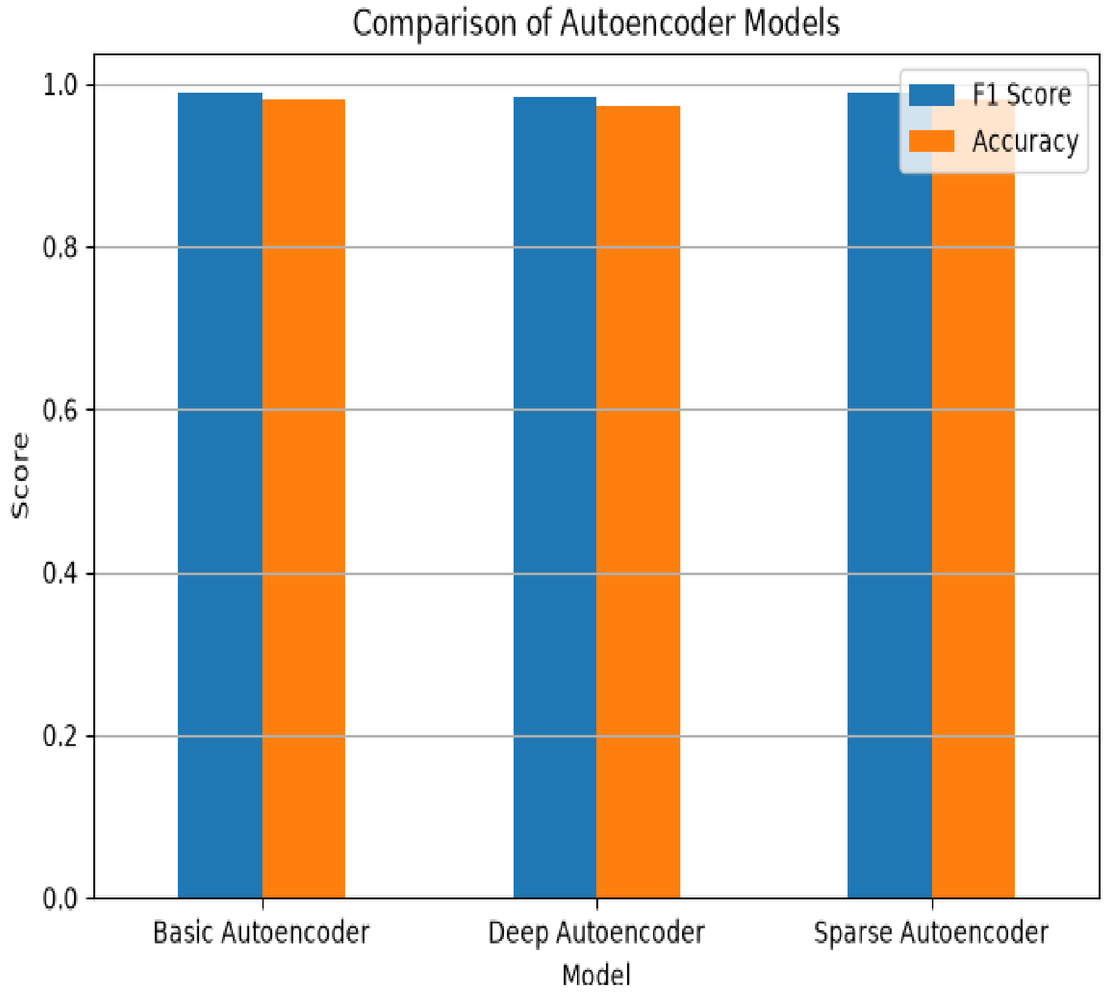
Key Findings

- Precision remained consistently high across all thresholds.
- F1 Score achieved the best balance around the selected threshold.
- Recall decreased as the threshold increased.
- The selected threshold provided reliable anomaly detection performance.



Comparison of Autoencoder Performance

- The highest F1 Score and Accuracy was obtained by using the Basic Autoencoder.
- Sparse Autoencoder was close to the performance of Basic Autoencoder.
- Deep Autoencoder gave competitive results with a slight drop in Accuracy.
- The three models showed a good performance in terms of anomaly detection.



Key Findings

- Basic Autoencoder achieved the highest F1 Score and Accuracy.
- Sparse Autoencoder showed performance close to the Basic Autoencoder.
- Deep Autoencoder demonstrated strong anomaly detection with slightly lower Recall and Accuracy.
- All three models achieved excellent performance for anomaly detection.

Thank you!

Group 3

SEP740 Deep Learning

McMaster University | Engineering